Anteater Chess v1.0

Produced by: Vishal Sinha, Ayush Chanda, Sahil Kulkarni, Oliver Engles, Brandon Hernandez, Shreshth Sharma, Rick Sharma

April 13, 2026

Table of Contents

Glossary **1**
1 Software Architecture Overview **2**
1.1 Main Data Type And Structures 2
1.2 Major Software Components 3
1.3 Modular Interface 4
1.4 Overall Program Control Flow 5
2 Installation **7**
2.1 System Requirements 7
2.2 Setup and Configuration 7
2.3 Uninstalling 8
3 Documentation of Packages, Modules, and Interfaces **9**
3.1 Detailed Description of Data Structures – ayush 9
3.2 Detailed Description of Functions and Parameters – ayush 12
3.3 Detailed Description of Input and Output Format - 15
4 Development Plan and Timeline **16**
4.1 Partitioning of tasks 16
4.2 Team Member Responsibilities 16
4.3 Weekly Milestone Schedule 17
Copyright **18**
Index(need to be fixed) **19**

Glossary

Term	Definition
Ant	An alternate name for a pawn in Anteater Chess.
Ant Eating	A special move unique to Anteater Chess where the anteater piece captures one or more adjacent pawns (ants) in a single move.
Anteater	A new chess piece unique to Anteater Chess. It moves one step in any direction but can only capture ants (pawns), potentially eating multiple ants in one move.
Bishop	A classic chess piece that moves diagonally any number of squares.
Board	The 10×8 grid on which Anteater Chess is played, with columns labeled A–J and rows labeled 1–8.
Castling	A special move involving the king and a rook, performed under specific conditions.
Check	A situation in which a player's king is under threat of capture on the opponent's next move.
Checkmate	A game-ending situation in which a player's king is in check and has no legal move to escape.
Computer Player	The automated opponent controlled by the Anteater Chess program.
En Passant	A special pawn capture move that can occur immediately after an opponent moves a pawn two squares forward.
File	A vertical column on the chess board, labeled A through J.
King	The most important chess piece; the game ends when the king is checkmated.
Knight	A chess piece that moves in an L-shape: two squares in one direction and one square perpendicular.
Legal Move	A move that conforms to all rules of Anteater Chess.
Move Log	A human-readable text file recording all moves made during a game.
Pawn	A chess piece (also called an ant) that moves forward one square at a time (two from the starting position) and captures diagonally.
Promotion	The transformation of a pawn into another piece (usually a queen/user’s choice) upon reaching the far end of the board.
Queen	The most powerful classic chess piece; moves horizontally, vertically, or diagonally any number of squares.
Rank	A horizontal row on the chess board, numbered 1 through 8.
Rook	A chess piece that moves horizontally or vertically any number of squares.
Square	A single cell on the chess board, identified by a letter (file) and number (rank), e.g., F4.
Stalemate	A game-ending situation in which a player has no legal move available to them and they are not in a check/checkmate. Can also occur when there is insufficient material for both sides (but the latter is more complicated).

1 Software Architecture Overview

1.1 Main Data Type And Structures

Chess board: data structure to represent the chess board. 2 dimensional array of the data type named space. [1]
Space: data structure that represents the fixed space on the chess board. Its property includes a pointer to a chess piece (can be null if there is no chess piece on the space), and it has the property of rank and file. Also contains the property of color. [1]
Rank and File: properties describing the fixed coordinates of the space. Rank and File are of data type enum to help us convert the Files “A-J” into numbers and the ranks 8-1 into a 0-7. [1]
Color: is of data type enum because it can only be black or white.
Piece: data structure used to represent the pieces in chess. Its properties include color, Piecetype, and moveset. [1]
Piecetype: of enum data type and can only be 1 of pawn/ant, anteater, king, queen, rook, bishop, and knight.
Moveset: a pointer to a function which returns an array of all possible locations the piece can move to. The locations are of type string. For example, a location can be “A5”(rank 5 and file A).
Logs: a data structure that stores a chronological list of game events. Each added entry records the move done by the player including details such as pieces moved, new positions, captures, and anything important.

Outline of Data Structure Hierarchy

1. **ChessBoard**
    - **Spaces**
        - **Piece***
        - **Rank**
        - **File**
        - **Color**
2. **Piece**
    - **PieceType**
    - **Moveset**
    - **Color**

1.2 Major Software Components

The program is divided into four different subsystems, each responsible for a distinct area of functionality [1] [2].

Game Startup/Manager:

- Allowing user to load game
- Showing the initial boardstate
- Allowing player to choose game mode (player vs player or player vs computer)
- Allowing player to choose computer difficulty

- Allowing player to choose side(black or white)

Chess Moves + Board State Manager:

- Gets moves from user
  - Checks if the move is legal
  - Check if it is a check, checkmate, capture, stalemate, promotion, en-passant, normal move to empty space
- Applies the moves
  - If capture remove the piece at the destination space(make it null) and place the capturing piece at the destination space
- Change the state of the board
  - If checkmate, end the game
  - If stalemate end the game
  - If undo was pressed redo the move/bring back pieces if captured

GameLog manager:

- Keeps a running check of the moves made [4]
- Updates log if undo is pressed
- Refreshes if a new game is started

Computer/AI Opponent:

- Calculates move for the computer
- Applies the move for the computer

Module Hierarchy Diagram

1. **Main Program**
   a. **Game Manager**
      i. **StartGame()**
         LoadGame()
         SaveGame()
            a. **Choosing difficulty/side, computer difficulty**
   b. **Game Engine**
      i. **GetPlayerMove()**
      ii. **IllegalMoveCheck()**
         MakeMove()
            a. **isCheckMate()**
               isStaleMate()
      iii. **UndoMove()**
      iv. **PrintBoard()**
   c. **AI**
      i. **CalculateMove()**
   d. **Game Log manager**

i. **AddMoveToLog()**
    1. **UndoMove()**
    2. **PrintLog()**

1.3 Modular Interface

API Function signatures for each major module are listed below. Full prototypes and explanations appear in Section 3.2. [1]

Game Manager (used by main program):

```
void StartGame(int mode, enum side, int difficulty); // starts the game
void LoadGame(char filename[]); // Loads the game saved on the file
void SaveGame(char filename[]); // Saves the game on a gamefile
```

Game Engine (used by Game Manager, AI, and Game Log Manager):

```
char GetPlayerMove() // should get the move the player wants to make via
printf and scanf, example: Bf3 indicates Bishop to f3
int IllegalMoveCheck() // checks if the move is valid or not and returns 1
if valid and 0 if invalid
int MakeMove(Piece piece, char startingCoordinate, char finalCoordinate) //
makes the move on the chess board, updates board by removing pieces and
moving them or promoting them etc
bool isCheck(space board [8][10]) // Checks if the current player is in
check
bool isCheckMate(space board [8][10]) // returns true if a player is in
checkmate
bool isStaleMate(space board [8][10]) // returns true if a player is
stalemated
void UndoMove(space board [8][10]) // undoes the most recent move or the
two most recent moves (one from player one from computer)
void PrintBoard(space board [8][10]) // prints the chess board
```

AI (Used by Game Engine and GameLog manager):

```
char CalculateMove() // returns the calculated move the engine makes
```

Game Log Manager (used by Main program):

```
void AddMoveToLog() // Adds the most recent move to the logs
void UndoMove() // depending on game mode and side undoes the most recent
move for the player
void PrintLog() // prints the game log history
```

1.4 Overall Program Control Flow

The general control flow is derived from the project specification [1] and implemented in main.c:
Main Program Flow

1. **Program Starts/Game initialization**
    - Welcome User and allow them to choose:
    - New game
        - Prompts Player to choose game mode (Player vs Player || Player vs Computer)
        - Prompts Player to choose side and computer difficulty
    - Load Game
        - Loads a saved gamefile
    - Initializes board and gamelog
    - Print initial board
2. **Main GameLoop (repeat until checkmate / stalemate / draw / exit)**
    a) Player's Move
        - GetPlayerMove() → IllegalMoveCheck() → re-prompt if invalid
        - MakeMove() → update board → PrintBoard() → AddMoveToLog()

    b) Check Game State
        - isCheck() && isCheckMate() && isStaleMate() + Draw logic to see game state → end game if terminal

    c) AI's/Second Player's Move (based on game mode)
        i) Player vs Player: Repeat step (a) for second player
        ii) Player vs AI mode: CalculateMove() → MakeMove() → update board, update log → PrintBoard() → AddMoveToLog()

    d) Miscellaneous(undo/menu + quit/save game + exit)
        - Undo: UndoMove() → Update board state + log + move order
        - Quit: prompt user if they want to save game → saveGame() or exit
        - New game: start new game and re-run initialization

    e) Game Ends
        - Display Winner/Result
        - Display Game log
        - Prompt User if they want a new game or to just exit

2 Installation

2.1 System Requirements

To ensure Anteater Chess program will run correctly on your device, the following requirements must be met: [3][5]

- Operating System: Windows, macOS Ventura or later, or Linux
- Processor: Intel I3 or equivalent (minimum) [1]
- Memory: 4 GB minimum
- Storage: At least 200 MB of free disk space
- Compiler: GCC Compiler
- Build Tool: GNU Make [1]
- Internet: For updates (optional)

2.2 Setup and Configuration

Please refer to these instructions when installing Anteater Chess: [1]

Step 1: Download Program

Obtain source code

- Can be done through Canvas submission files

Step 2: Extract Files

If the program file is compressed, extract it using the following:

```
unzip anteater_chess.zip
cd anteater_chess
```

Step 3: Compile the Program

Use the following makefile to generate executable file “AnteaterChess”:

```
make AnteaterChess
```

Step 4: Run the Program

Run the following line to start the program:

```
./AnteaterChess
```

Step 5: Follow the Directions on Screen

To start a game:

1. Choose your difficulty level(Easy/Medium/Hard)
2. Pick the side you wanna play as(Black/White)
3. Select if you want to play vs AI or Human
4. Hit Enter to Start

2.3 Uninstalling

Please refer to these instructions when uninstalling chess:

Run the following in terminal:

1. Reach installation directory using the following command:

```
cd anteater_chess
```

2. Remove all files using:

```
rm -rf anteather_chess
```

For Windows:

Right click and press “Delete” on Windows program

If asked to “Confirm”, click “Yes” to finish Delete

3 Documentation of Packages, Modules, and Interfaces

3.1 Detailed Description of Data Structures

- Critical snippets of source code: Data structures that will be implemented in our chess program: [1]
    - Enum
    - Union
    - 2D-Arrays
    - Structs
    - Possible chance of linked lists (we have not decided yet)
- Our team has to decide to use or implement the chess board as a 2D array and the reasons for this are:
    - The chest board is much simpler to implement using a 2D array opposed to using a linked list
    - We can access the positions of the chess board using traditional array indexing
    - A 2D array is also very easy to use and much easier to understand for coders and users, when explained in documentation
    - We are thinking of using having it set as chessBoard[8][10] with 8 ranks and 10 files
    - We have also decided the cons of linked lists outweigh the pros of using a 2D array
    - If we plan to use linked lists for the chess board, we may have to use it has an array of linked lists
        - Indexing can be hard or accessing positions of the struct pieces
        - It is very hard to move across rows if we use the columns as linked lists
        - It is very hard to move across columns if we use the rows as linked lists
        - Ideal way of iterating across linked list is using next pointers, and if we define doubly linked list, we have to use previous and next pointers
- Header file snippets
    - moveFunctions.h

```c
#ifndef moveFunctions
#define moveFunctions

//moves sets of different functions
char *moveSet(Piece, space board[8][10]);
void moveKing(Piece, space board[8][10]);
void moveQueen(Piece, space board[8][10]);
void movePawn(Piece, space board[8][10]);
void moveAnteater(Piece, space board[8][10]);
void moveKnight(Piece, space board[8][10]);
void moveRook(Piece, space board[8][10]);
void moveBishop(Piece, space board[8][10]);

#endif
```

- gameEngine.h

```c
#ifndef gameEngine
#define gameEngine

#include <stdbool.h> //import this header file to declare functions of boolean return type

//should get the move the player wants to make via printf and scanf
char GetPlayerMove();

int IllegalMoveCheck(); //checks if the move is valid or not and returns 1 if valid and 0 if invalid

int MakeMove(Piece piece, String startingcoordinate, String finalCoordinate); //makes the move on the chess board
//Updates board by removing pieces and moving them or promoting them etc

bool isCheck( space board [8][10] ); //Checks if the current player is in check

bool isCheckMate( space board [8][10] ); //returns true if a player is in checkmate

bool isStaleMate( space board [8][10] ); //returns true if a player is stalemated

void undoMove( space board [8][10] ); //undoes the most recent move or the two most recent moves (one from player one from computer)

void PrintBoard( space board [8][10] ); //prints the chess board

char CalculateMove() // returns the calculated move the engine makes

#endif
```

- logManager.h

```c
#ifndef moveFunctions
#define moveFunctions

//moves sets of different functions
char *moveSet(Piece, space board[8][10]);
void moveKing(Piece, space board[8][10]);
void moveQueen(Piece, space board[8][10]);
void movePawn(Piece, space board[8][10]);
void moveAnteater(Piece, space board[8][10]);
void moveKnight(Piece, space board[8][10]);
void moveRook(Piece, space board[8][10]);
void moveBishop(Piece, space board[8][10]);

#endif
```

- main.c (runner file where all header files are linked)

main.c

```c
#include <stdio.h>
#include <gameEngine.h>
#include <logManager.h>
#include <math.h>
#define LENGTH 80

//generate enum data structures to evetually convert these strucutres into numerical data
typedef enum { A,B,C,D,E,F,G,H,I,J } File;
typedef enum { R1,R2,R3,R4,R5,R6,R7,R8 } Rank;
typedef enum { WHITE, BLACK } Color; //0 --> white, 1 ---> black
typedef enum { PAWN, ANTEATER, KING, QUEEN, ROOK, BISHOP, KNIGHT } PieceType;

//declare a 2D array to populate the spaces and to generate a chess board

struct Piece {
    Color color;
    PieceType type;
    char []moveSet;
}

struct Space {
    Piece *piece; //pointer to a piece structure
    Rank rankNum;
    File fileNum;
    Color color

}
```

main.c

```c
//make instances of the piece structure and declare them
//definition of its attributes will be made later in the code
Piece King1;
Piece King2;
Piece Queen1;
Piece Queen2;
Piece Anteater;
Piece Bishop1;
Piece Bishop2;
Piece Knight1;
Piece Knight2;
Piece Rook1
Piece Rook2;

//function to store the piece the user selected to make a move on
Piece storePiece(Piece piece);


int main()
{
    int difficulty;
    int gameType;

    printf("Welcome to Anteater Chess! \n");

    printf("Select the type of game you want to play \n");
    printf("1. Player versus Player");
    printf("2. Player versus Computer");
    scanf("%d", &gameType);

    if (game == 1) {
        //section of the code if player wants to play another player
    }
    //means the gameType has an integer value of 2 so the player wants to play the computer
    else {
        printf("Choose difficulty level, 1 for Easy, 2 for Medium, 3 for Hard): ");
        scanf("%d", &difficulty);

        int gameChoice;
        printf("Choose side, 0 for white, 1 for black: ");
        scanf("%d", &gameChoice);

        while ((gameChoice != 1) || (gameChoice != 0)) {
            printf("Choose side, 0 for white, 1 for black: ");
            scanf("%d", &gameChoice);
        }
        side = (sideChoice == 1) ? WHITE : BLACK;
    }
```

```c
    Space chessBoard[8][10];

    PrintBoard(chessBoard);

    bool gameDone = false;

    char winner;

    while (gameDone == false) {

        char playerMove[LENGTH]; //can take a max move input of 80 characters
        playerMove = GetPlayerMove();

        if (IllegalMoveCheck(chessBoard)) {
            makeMove(storePiece);
            AddMoveToLog();

            //checks for checkmate
            if (isStaleMate(chessBoard)) {
                printf("It is a draw between the 2 players. The game is finished");
                gameDone = true;

            }

            if(isCheckMate(chessBoard)) {
                printf("The game is finished. The winner is %c", winner);
                gameDone = true;

            }
        }

        StartGame();

        LoadGame();
    }

    //After the game is finished either with a win or a draw
    //SaveGame(filenameInput);

    PrintLog();

    return 0;
}
```

3.2 Detailed Description of Functions and Parameters

- Function prototypes and brief explanations
- Have prototypes of functions in a header files
    - moveFunctions.h
    - gameEngine.h
    - logManager.h
- gameEngine:

```
#ifndef gameEngine
#define gameEngine

#include <stdbool.h> //import this header file to declare functions of
boolean return type

//should get the move the player wants to make via printf and scanf
char GetPlayerMove();
int IllegalMoveCheck(); //checks if the move is valid or not and returns 1
if valid and 0 if invalid
int MakeMove(Piece piece, String startingcoordinate, String
finalCoordinate); //makes the move on the chess board
//Updates board by removing pieces and moving them or promoting them etc
// "String" type used in the function parameters represents a character
array (char[]), these store board coordinates such as "E2".
bool isCheck( space board [8][10] ); //Checks if the current player is in
check
bool isCheckMate( space board [8][10] ); //returns true if a player is in
checkmate
bool isStaleMate( space board [8][10] ); //returns true if a player is
stalemated
void UndoMove( space board [8][10] ); //undoes the most recent move or the
two most recent moves (one from player one from computer)
void PrintBoard( space board [8][10] ); //prints the chess board
char CalculateMove() // returns the calculated move the engine makes

#endif
```

<u>Explanation for gameEngine.h functions prototype:</u> For the header file above with the function prototypes, a lot of them are in boolean return type like “isCheck”, “isCheckMate”, “isStaleMate” because these functions can the status if one of the 2 chess players is in danger of losing the chess game. The status’s value can be best understood using the boolean data type, and these boolean functions along with “UndoMove” and “PrintBoard” pass in a 2D array of the type space. A 2D array needs to be passed in, so the void and boolean functions can perform operations to manipulate the structure or individual elements of the 2D array board. The

parameter should be of type space because our team decided that each element should be a struct data structure with the name “space”. The Undo operation restores the board to its previous state by reversing the most recent move done. Undo will reposition pieces, restore captured pieces if any, and update turn order. Calculate move will call upon the AI module which will evaluate all possible moves based on the current state of the board. It will use a search based approach to find the best possible move and then pass this information to the game engine for execution [1][2].

- logManager.h:

```
#ifndef logManager
#define logManager
#define LENGTH 80

void AddMoveToLog() // Adds the most recent move to the logs
//depending on game mode and side undoes the most recent move for the
player

void UndoMove( char filename[LENGTH])
void PrintLog( char filename[LENGTH]) //prints the game log history

#endif
```

<u>Explanation for function prototypes in logManager.h:</u> For the header file logManager.h, all function prototypes are written and declared in the void type. These void functions don’t return anything, but they are used to keep record and track of all the moves made on the 2d array of the return type “space” [1].

- moveFunctions.h

```
#ifndef moveFunctions
#define moveFunctions

//moves sets of different functions
char *moveSet(Piece, space board[8][10]);
void moveKing(Piece, space board[8][10]);
void moveQueen(Piece, space board[8][10]);
void movePawn(Piece, space board[8][10]);
void moveAnteater(Piece, space board[8][10]);
void moveKnight(Piece, space board[8][10]);
void moveRook(Piece, space board[8][10]);
void moveBishop(Piece, space board[8][10]);

#endif
```

Explanation: These function declarations are the function prototypes of different pieces. All of the void functions move the piece given as a parameter, on the 2d array board through indexing and use of other functions. These void functions also make sure the operations and indexing are done on the correct or valid types of piece structures. On top of these void function prototypes, there is another prototype for a function that returns a pointer to a sequence of characters, that returns the sequence of characters describing the possible moves of the piece for specific pieces [1].

3.3 Detailed Description of Input and Output Format -

3.3.1 Move input from user:

Anteater Chess supports multiple input methods depending on the implementation stage. Our terminal mode allows the players to enter moves using standard inputs (via printf and scanf). In GUI mode the player selects a piece and destination using their mouse and its interaction with the environment. Regardless of how the move is made, the move detected is ultimately always converted into board coordinates and passed to the engine for execution [1].

Player moves are captured through mouse interactions with the GUI(graphical user interface). The player clicks on a piece they wish to move. The program identifies the board coordinate of the selected piece and highlights all legal destination squares via getcoordinate(). The player then clicks on the highlighted destination squares the program then decides the move from the available legal moves and passes the chosen one to makemove(). If the player clicks on an invalid target or square the selection resets and there is no move recorded. If during the move there is an available pawn promotion an additional prompt appears asking the player to select the piece type they wish to promote the pawn to.

3.3.2 Move output in the log file:

All moves performed during the game session are recorded in a log file using standard algebraic notation, consistent with FIDE conventions [4]. The log is written by AddMovetoLog() and this function stores it in a plain text file. The game log is maintained in memory during gameplay and is optionally written to a text file for persistence and later review.

For example:

1. e4 e5
2. Nf3 Nc6
3. Bb5 a6

//A coordinate is represented as a two-character string consisting of a file (A-J) followed by a rank (1-8)

For special moves it is documented as follows:

Move type	Notation
Castling (kingside)	o-o
Castling (queenside)	o-o-o
En pessant	Standard capture notation to destination square
Pawn promotion	Destination square + piece promoted to (Bb6=Q)
Check	+ In front of pieces move square (+Bb6)
Checkmate	# In front of pieces move square (#Bb6)

4 Development Plan and Timeline

4.1 Partitioning of tasks

The codebase is structured around six independently developable modules plus shared infrastructure. The table below maps each module to its primary deliverable and its key dependencies [1].

Module / Task	Key Deliverables	Depends On
Main Program (entry point & game loop)	main.c, Makefile, game startup menu, difficulty/side selection	All modules
Game Engine (move execution & state)	gameEngine.h/.c — MakeMove, illegalMoveCheck, isCheck, isCheckMate, isStaleMate, UndoMove	Chess Objects, Move Functions
Move Functions (piece movement rules)	moveFunctions.h/.c — moveKing, moveQueen, movePawn, moveAnteater, moveKnight, moveRook, moveBishop	Chess Objects
Chess Objects (data structures)	Piece, Space, ChessBoard structs; enum definitions for Color, PieceType, File, Rank	(none — foundational)
AI / Strategy	CalculateMove() — minimax or heuristic evaluation; difficulty levels (easy / medium / hard)	Game Engine, Move Functions
Game Log Manager	logManager.h/.c — AddMoveToLog, PrintLog, UndoMove; algebraic notation output to text file	Game Engine
GUI / Display	PrintBoard (ASCII fallback); graphical SDL/GTK board rendering; highlighting of legal moves	Chess Objects, Game Engine

Additional shared tasks not owned by a single module:

- Integration testing — all team members; scheduled end of Week 3 and throughout Week 4.
- Documentation — each member documents their own module; Vishal & Shreshth maintain the top-level docs.
- Version control (git) — all members commit to the team repository on the course servers daily.
- Tournament preparation — AI tuning and illegal-move hardening prioritized in Week 4.

4.2 Team Member Responsibilities

Member	Primary Module	Secondary / Support Role	Week 4 Focus
Vishal Sinha	Main Program & Game Manager	Documentation lead, project coordination	Integration, final docs
Ayush Chanda	Data Structures & Chess Objects	Function prototypes (gameEngine.h review)	Code review, bug fixes
Sahil Kulkarni	Game Engine	Move legality & check/checkmate logic	Edge-case testing (castling, en passant, ant eating)
Oliver Engles	Move Functions	Piece movement rules for all 7 piece types	Anteater move edge cases, promotion logic
Brandon Hernandez	GUI / Display	ASCII board (Wk 1–2), GUI board (Wk 3–4)	GUI polish, game-end screen
Shreshth Sharma	AI / Strategy	Minimax algorithm, difficulty scaling	AI tuning for tournament
Rick Sharma	Game Log Manager	Algebraic notation, undo/redo support	Save/load game, log output

Note on the tournament: At the close of Week 4, all teams compete in the Anteater Chess tournament on the course lab servers. An illegal move immediately forfeits the game, so correct rule implementation takes priority over advanced AI. Teams are advised to complete and harden the move-legality checker before investing time in AI heuristics.

Copyright

Copyright © 2026 EECS 22L Team, University of California, Irvine. All rights reserved.

This software and associated documentation ("Anteater Chess") were developed as a student project for EECS 22L: Software Engineering Project in C Language, University of California, Irvine, Spring 2026.

Permission is granted for educational and non-commercial use within UCI. Redistribution or use of this software outside the context of this course requires explicit written permission from the authors and the course instructor.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED. IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES, OR OTHER LIABILITY ARISING FROM USE OF THIS SOFTWARE.

The rules of standard chess are in the public domain. The Anteater Chess variant rules are defined by Prof. Rainer Dömer for EECS 22L, UCI.

References

[1] R. Dömer, *EECS 22L: Project 1 — The Game of Anteater Chess*, Department of Electrical Engineering and Computer Science, University of California, Irvine, March 30, 2026.

[2] R. Dömer, *EECS 22L: Chess User Manual Grading Criteria*, University of California, Irvine, March 30, 2026.

[3] C. Yuan and R. Dömer, *EECS 22L: Chess Software Specification Grading Criteria*, University of California, Irvine, April 5, 2026.

[4] FIDE, *Laws of Chess*, Fédération Internationale des Échecs, effective January 1, 2023. [Online]. Available: https://www.fide.com/FIDE/handbook/LawsOfChess.pdf

[5] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

Index

Page numbers refer to the complete Chess_SoftwareSpec.pdf document, with this supplement appended as pages 17–19.

AddMoveToLog(), 7, 13, 15
AI / Strategy module, 3–4, 7
Algebraic notation, 15
Ant eating, Glossary (p. 1), 15
Anteater piece, Glossary (p. 1)
Board (ChessBoard struct), 2, 7, 9
CalculateMove(), 4, 7, 12–13
Castling, Glossary (p. 2), 15
Check / Checkmate, Glossary (p. 2), 4, 7
Chess_Alpha.tar.gz, 6, 18
Chess_SoftwareSpec.pdf, 6
Chess_UserManual.pdf, 6, 18
Chess_V1.0.tar.gz, 6, 18
Color (enum), 2, 9
Control flow, program, 4–6
Copyright, 17
Data structures, 2, 7–11
Development plan, 16–18
Difficulty levels (AI), 3, 7, 18
En passant, Glossary (p. 2), 15
File (enum), 2, 9
Game Engine module, 3–4
Game Log Manager, 3–4, 7
Game Manager module, 3–4
gameEngine.h, 4, 7, 10, 12–13
GetPlayerMove(), 4, 7, 12
git (version control), 6, 18
GUI / Display, 3, 5, 7, 18
Header files, 7, 10–14
Index, 9, 19
Installation, 5–6
isCheck(), 4, 7, 12–13
isCheckMate(), 4, 5, 7, 12–13
isStaleMate(), 4, 5, 7, 12–13
King (piece), Glossary (p. 2)
Legal move, Glossary (p. 2)
LoadGame(), 3–4, 7
logManager.h, 7, 10, 13–14
main.c, 9, 11
Makefile, 5, 18
MakeMove(), 4–5, 7, 12–13
Module hierarchy, 3–4
Move log, Glossary (p. 2), 3, 15
moveFunctions.h, 7, 10, 14
Pawn / Ant (piece), Glossary (p. 1–2)
Piece (struct), 2–3, 9
PieceType (enum), 2–3, 9
PrintBoard(), 4, 7, 12
PrintLog(), 4, 7, 13
Promotion (pawn), Glossary (p. 2), 15
Rank (enum), 2, 9
References, 19
SaveGame(), 3–4, 7
Space (struct), 2, 9
Stalemate, Glossary (p. 2), 4, 5
System requirements, 5
Team member responsibilities, 16–18
Timeline / schedule, 18
Tournament, 18
UndoMove(), 3–4, 7, 12–13
Uninstalling, 6